



**university of
 groningen**

**faculty of science
and engineering**

Nordic Wind Power Forecasting

Bogdan Sandoiu(S5549329)

Abstract

Accurate short-term wind power forecasting is essential for electricity markets with a high share of renewable energy. This paper addresses the task of forecasting hourly wind power production up to two days ahead for multiple Norwegian bidding zones, following the data availability constraints of the WindAI challenge. The dataset includes historical wind power production, observed meteorological variables, and numerical weather prediction forecasts.

Model families are evaluated, including statistical time series models (SARIMA and SARIMAX), recurrent neural networks based on gated recurrent units (GRUs), and Transformer-based architectures. While statistical and recurrent models provide reasonable baselines, Transformer models consistently achieve lower forecast error, particularly at longer prediction horizons, due to their ability to capture long-range temporal dependencies through attention mechanisms.

Overall, the results demonstrate that two-day-ahead wind power forecasting is feasible under realistic operational constraints and that attention-based models offer clear advantages over recurrent architectures when combined with appropriate preprocessing and feature engineering.

Contents

1	Introduction	2
1.1	Motivation and challenges	2
1.2	Contributions	2
2	Data	2
3	Data Preprocessing and Feature Engineering	3
3.1	Feature engineering for deep learning	3
3.2	Model-specific preprocessing	4
3.2.1	Preprocessing for statistical models	4
3.2.2	Preprocessing for deep learning models	4
3.3	Window construction	4
4	Methods	4
4.1	Statistical baseline models	4
4.2	Deep learning models	5
4.2.1	Recurrent baselines and transition to Transformers	5
4.3	Training procedure	6
5	Results	6
5.1	Statistical baseline models	6
5.2	Deep learning models	6
5.3	Transformer models: masked vs. unmasked decoder	7
6	Discussion	8

1 Introduction

Wind power plays an important role in modern electricity systems, especially in regions such as the Nordic countries where wind energy penetration is high. However, wind power production is highly variable and difficult to predict, which creates challenges for power system operation and electricity market planning. Transmission system operators therefore rely on short-term forecasts to schedule production and maintain grid stability.

This work is conducted in the context of the WindAI challenge [1]. The objective is to forecast hourly wind power production up to two days ahead for the Norwegian bidding zones NO1 to NO4, as illustrated in Figure 1. Forecasts must be generated using only information that would be available at prediction time, including historical power production, observed weather data, and numerical weather prediction forecasts.

1.1 Motivation and challenges

Short-term wind power forecasting involves several well-known challenges. First, wind power time series are non-stationary, mainly because of seasonal and diurnal cycles, changing weather conditions, and long-term structural changes such as increasing installed wind capacity. These factors lead to changes in the statistical properties of the data over time, making modeling more difficult.

Second, forecast accuracy generally decreases as the prediction horizon increases. This is largely because wind power forecasts rely on numerical weather prediction (NWP) models, whose uncertainty grows further into the future. As a result, errors in weather forecasts propagate into wind power predictions, especially for longer horizons.

Third, different bidding zones are characterized by different wind regimes, influenced by geography, terrain, and local climate conditions. This spatial variability makes it difficult to develop models that generalize well across regions without region-specific adaptation.

Recent deep learning approaches, including Transformer-based models, have shown promising results in sequence modeling tasks due to their ability to capture long-range temporal dependencies using attention mechanisms [2]. However, in time series forecasting, Transformers do not consistently outperform simpler models, particularly when strong seasonal patterns dominate or when the amount of available data is limited [3]. This raises the question of whether the increased model complexity is justified for operational wind power forecasting tasks.

1.2 Contributions

This paper investigates the effect of model complexity and attention mechanisms on two-day-ahead wind power forecasting. The main contributions are:

- An end-to-end forecasting pipeline that follows the WindAI data availability constraints.
- A comparison between statistical models, recurrent neural networks, and Transformer-based models.
- Experimental evidence that Transformers outperform recurrent neural network baselines.

2 Data

This study uses the WindAI dataset [4], which provides hourly wind power production and meteorological data for the Norwegian bidding zones NO1–NO4. The dataset is designed for short-term forecasting under realistic operational conditions, combining observed weather variables (nowcasts) with ensemble numerical weather prediction (NWP) inputs to support multi-step forecasts up to 65 hours ahead [4].

Wind power production is reported as aggregated hourly values per bidding zone and measured in megawatts (MW). Meteorological data are available at the wind-park level and include both observed variables and ensemble forecasts generated by the Meteorological Ensemble Prediction System (MEPS), operated by MET Norway (the Norwegian Meteorological Institute). MEPS provides multiple forecast realizations that enable the quantification of predictive uncertainty [4]. Table 1 summarizes the data sources used in this work, while Figure 1 illustrates the geographical locations of the considered bidding zones.

Table 1: Overview of datasets used in the WindAI challenge [4]

Dataset	Description
Wind power production	Hourly aggregated wind power generation per bidding zone (NO1–NO4), measured in megawatts (MW) [4].
Nowcasting weather data	Observed weather variables at wind-park level (e.g., wind speed, wind direction, temperature, pressure, humidity, precipitation), available up to forecast issuance time [4].
Weather forecasts (NWP)	Ensemble numerical weather prediction forecasts from the MET Norway MEPS system, providing meteorological variables up to 65 hours ahead [4].
Wind park metadata	Information about wind park locations, bidding zone assignments, and operational properties [4].



Figure 1: Nordic power market bidding zones considered in the WindAI challenge.

3 Data Preprocessing and Feature Engineering

All experiments use a chronological split into training, validation, and test sets. For a forecast issued at time t , only data available at or before time t are used as model inputs. Any statistics used for feature scaling are computed using the training set only and then applied unchanged to the validation and test sets.

3.1 Feature engineering for deep learning

The features described in this subsection are used only by deep learning models. Statistical models rely on raw or minimally processed wind power time series.

Wind direction encoding. Wind direction is a circular variable, meaning that directions close to 0° and 360° correspond to nearly identical physical states. To avoid discontinuities arising from this circularity, wind speed and wind direction are transformed into two horizontal wind components. Given a wind speed $s(t)$ and wind direction $\theta(t)$, the components are computed as:

$$u(t) = s(t) \cos(\theta(t)), \quad v(t) = s(t) \sin(\theta(t)). \quad (1)$$

Here, the wind direction $\theta(t)$ is expressed in radians. This transformation is applied to both observed and forecast wind variables, after which the original wind speed and direction are replaced by the corresponding horizontal components.

Cyclic time features. Daily and yearly seasonal patterns are represented using cyclic time encodings. Four time-based

features are added:

$$\text{Day}_{\sin} = \sin\left(\frac{2\pi \cdot t}{\text{one day}}\right), \quad \text{Day}_{\cos} = \cos\left(\frac{2\pi \cdot t}{\text{one day}}\right),$$

$$\text{Year}_{\sin} = \sin\left(\frac{2\pi \cdot t}{\text{one year}}\right), \quad \text{Year}_{\cos} = \cos\left(\frac{2\pi \cdot t}{\text{one year}}\right).$$

These features allow the models to learn smooth daily and yearly seasonality without introducing artificial jumps between consecutive time steps.

Lagged wind power features. To provide short-term historical context, lagged values of wind power production are included. For each time step t , lag features are created as:

$$y(t-1), y(t-2), \dots, y(t-7),$$

where $y(t)$ denotes the wind power production at time t . These lag features help the models capture recent trends and autocorrelation in the data.

Weather ensemble summary features. Weather forecasts are provided by the MET Norway MEPS, which generates multiple forecast realizations for each weather variable at every forecast issuance time. Each ensemble member predicts future values of wind speed, wind direction, temperature, pressure, humidity, and wind gusts for multiple lead times.

To transform these ensemble forecasts into fixed-size inputs suitable for deep learning models, simple statistical summaries are computed across ensemble members. For each weather variable and forecast lead time, the ensemble mean and standard deviation are calculated. For example, the mean wind speed at time t is computed as

$$\text{mean}(t) = \frac{1}{N} \sum_{i=1}^N s_i(t),$$

where $s_i(t)$ denotes the wind speed predicted by ensemble member i , and N is the total number of ensemble members. The corresponding standard deviation is computed as

$$\text{std}(t) = \sqrt{\frac{1}{N} \sum_{i=1}^N (s_i(t) - \text{mean}(t))^2}.$$

For wind direction, the same encoding strategy described earlier is applied before computing ensemble statistics. Specifically, wind speed $s_i(t)$ and wind direction $\theta_i(t)$ from each ensemble member are first converted into horizontal wind components using sine and cosine transformations. The ensemble mean and standard deviation are then computed separately for the resulting horizontal wind components. This ensures consistency between observed and forecast wind representations and avoids discontinuities at the $0^\circ/360^\circ$ boundary. The resulting ensemble summary features capture both the expected future weather conditions through the ensemble mean and the associated forecast uncertainty through the ensemble spread. All ensemble statistics are computed using only information available at the forecast issuance time and are used exclusively as input features for the deep learning models.

3.2 Model-specific preprocessing

Different preprocessing pipelines are used for statistical models and deep learning models due to their different modeling assumptions.

3.2.1 Preprocessing for statistical models

The Seasonal Autoregressive Integrated Moving Average (SARIMA) and Seasonal Autoregressive Integrated Moving Average with Exogenous Variables (SARIMAX) models operate on univariate or low-dimensional time series and therefore employ a deliberately simple preprocessing pipeline. Hourly wind power data are reshaped from wide to long format and aggregated per bidding zone by summing production across all wind parks, yielding a single continuous time series for each zone.

To capture deterministic seasonality, cyclic time features are constructed from the timestamp. The same sine and cosine encoding technique used for feature engineering in the deep learning models is reused here to represent daily and yearly seasonal patterns in a smooth and continuous manner. These cyclic features are included as optional exogenous regressors for the SARIMAX model, while SARIMA relies solely on the wind power series.

After aggregation, the data are split by bidding zone and ordered chronologically. Missing values are removed, and only numerical inputs required by each model are retained. Stationarity is handled internally through differencing and seasonal differencing when required, and no normalization or scaling is applied. All inputs are strictly aligned in time to prevent information leakage.

3.2.2 Preprocessing for deep learning models

Deep learning models operate on multivariate inputs and use the engineered features described above. Weather variables are first aggregated from wind park level to bidding zone level by computing the mean across all wind parks within each zone.

Feature construction is performed separately for the training, validation, and test splits. To preserve causality at split boundaries, lagged features for the validation and test sets are computed using a short history from the preceding split. After feature construction, numerical features are standardized using statistics computed on the training set only. For a feature x , normalization is performed as

$$x_{\text{norm}} = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}},$$

where μ_{train} and σ_{train} denote the mean and standard deviation computed on the training data. The same transformation is then applied unchanged to the validation and test sets.

Cyclic time features based on sine and cosine encodings are not standardized, as their values are already bounded within the interval $[-1, 1]$.

3.3 Window construction

The processed time series is converted into supervised learning samples using sliding windows. A window represents a fixed-length snapshot of the recent past that the model observes in order to make a forecast.

For a bidding zone z and time t , the input window of length L is defined as:

$$X(z, t) = \{f(z, t - L + 1), \dots, f(z, t)\},$$

where $f(z, t)$ contains the standardized weather variables and engineered features available at time t . In practice, this window summarizes the evolution of weather conditions and power production over the previous L hours and serves as the model input.

The corresponding target is a multi-step forecast horizon:

$$Y(z, t) = \{y(z, t + 1), \dots, y(z, t + H)\},$$

which contains the true wind power production for the next H hours following time t .

Windows that overlap dataset boundaries or contain missing values are discarded to ensure consistency and causality. The same windowing strategy is used for all deep learning models so that differences in performance are attributable to model architecture rather than data preparation.

4 Methods

This section describes the forecasting models used in this work and explains how each architecture operates. All models are evaluated using identical temporal splits, input windows, and forecast horizons to ensure a fair comparison.

4.1 Statistical baseline models

Statistical models are included as reference baselines. They rely on linear assumptions and fixed temporal structures.

SARIMA. SARIMA extends the classical Autoregressive Integrated Moving Average (ARIMA) model, which represents the current value of a time series as a linear combination of past observations and past forecast errors after differencing to ensure stationarity. While ARIMA captures short-term temporal dependencies, it does not explicitly model seasonal patterns. SARIMA addresses this limitation by incorporating a seasonal autoregressive and moving-average structure with a fixed seasonal period of 24 hours, corresponding to daily cycles in wind power production. In this work, SARIMA is implemented using the SARIMAX framework without exogenous variables. The selected configuration combines a non-seasonal order $(2, 1, 2)$ with a seasonal order $(1, 1, 1, 24)$, allowing the model to capture both intra-day dynamics and daily repetition. As with ARIMA, forecasts are generated directly for the next H hours and evaluated using RMSE.

SARIMAX. SARIMAX further extends SARIMA by incorporating external explanatory variables that are known at forecast issuance time. In this project, sinusoidal time features representing daily and yearly cycles are included as exogenous regressors. These variables provide the model with explicit information about periodic structure that may not be fully captured by autoregressive terms alone. The autoregressive and seasonal orders are kept identical to those used in the SARIMA model to ensure a fair comparison. By combining past wind power observations with time-based external information, SARIMAX aims to improve forecast accuracy while maintaining a relatively simple and interpretable structure.

4.2 Deep learning models

Deep learning models use an input window of length $L = 336$ hours and predict an output horizon of length $H = 48$ hours. The input features include weather variables and engineered features (time encodings, wind components, and lag features) as described in Section 3.1.

For comparability, all deep learning models are evaluated using the same error metric, namely root mean squared error (RMSE). Models are trained using mean squared error (MSE) as the optimization objective, while evaluation is reported using RMSE for interpretability. While different architectures could benefit from task-specific training objectives, a common loss function is used here to reduce confounding factors and simplify comparison across models. The training loss at forecast issuance time t is defined as:

$$\mathcal{L}(t) = \frac{1}{H} \sum_{h=1}^H \left(y(t+h) - \hat{y}(t+h) \right)^2, \quad y(t+h), \hat{y}(t+h) \in \mathbb{R}. \quad (2)$$

At inference time, forecasts are generated autoregressively without access to future ground-truth values.

4.2.1 Recurrent baselines and transition to Transformers

The recurrent modeling approach was explored in several stages using GRU-based architectures. We initially trained a high-capacity GRU model (GRU_Deep) with the goal of capturing complex temporal dependencies. Although this model achieved very low training error, it exhibited strong overfitting (Figure 2), with validation performance degrading quickly. This indicates that the model capacity was too large relative to the effective information content of the data, leading to poor generalization. The architectural details of this model, as well as all other neural architectures used in this work, are summarized in Table 2.

To address the strong overfitting observed in the GRU_Deep model, we introduced a lower-capacity recurrent architecture (GRU_Weak). In addition, hyperparameter optimization was performed using Optuna [5], a Bayesian optimization framework, to tune key training and architectural parameters such as the learning rate, hidden size, dropout rate,

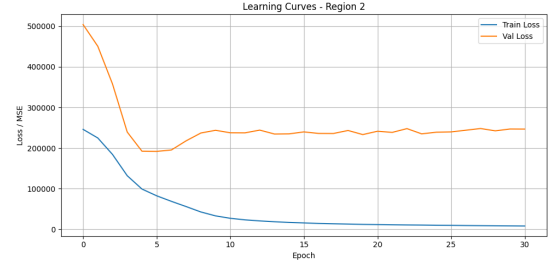


Figure 2: Training and validation loss curves for the GRU_Deep model, illustrating rapid overfitting despite strong optimization of the training objective.

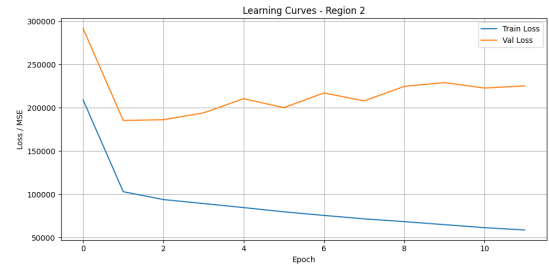


Figure 3: Training and validation loss curves for the GRU_Weak model, showing more stable convergence and reduced overfitting compared to the higher-capacity GRU_Deep architecture.

and regularization strength. Despite this tuning, the higher-capacity GRU_Deep model consistently exhibited unstable training dynamics and pronounced overfitting. In contrast, the reduced-capacity GRU_Weak model showed more stable convergence and improved validation performance, as illustrated by the training and validation loss curves in Figure 3. For this reason, GRU_Weak is retained as the primary recurrent baseline in this study (see Table 2).

Despite these improvements, both GRU-based models consistently converged toward predicting the mean or median level of the wind power signal. As shown in Figure 4, the recurrent models struggled to capture sharp ramps, peaks, and other abrupt temporal variations, instead favoring smooth average forecasts. This behavior suggests that, under the given data regime and training setup, recurrent architectures were unable to fully exploit long-range dependencies and complex interactions in the input features.

The observed limitations of recurrent models motivated the exploration of Transformer-based architectures. Transformers were evaluated using two decoder configurations: a masked (causal) decoder and an unmasked decoder, both described in Table 2. This comparison allows us to assess the effect of enforcing strict temporal causality in the decoder. As shown in the Results section, Transformer models substantially outperform the recurrent baselines, with the unmasked variant achieving the best overall forecasting performance.

Table 2: Summary of deep learning architectures (input length $L = 336$, forecast horizon $H = 48$).

Design choice	GRU_Weak	GRU_Deep	Transformer
Input	(L, F) window	(L, F) window	Encoder: (L, F) , Decoder: $(H, 1)$
Input regularization	GaussianNoise(0.1)	GaussianNoise(0.1)	None
Core layers	GRU $256 \rightarrow 128 \rightarrow 64$ (seq)	GRU $512 \rightarrow 512 \rightarrow 512 \rightarrow 256 \rightarrow 128$ (seq)	2 encoder blocks + 2 decoder blocks
Normalization	LayerNorm after each GRU	LayerNorm after each GRU	LayerNorm inside each block
Dropout	0.4, 0.3, 0.2 (+ head 0.3)	0.2 after each GRU	0.1 inside attention blocks
Regularization	L2 on GRU (10^{-3})	L2 on GRU (10^{-4})	AdamW weight decay (10^{-5}) if available
Horizon handling	Slice last H steps, then dense head	Slice last H steps, then dense head	Teacher forcing during training
Output head	TD Dense(64, ReLU) $\rightarrow$ Dense(32, ReLU) $\rightarrow$ Dense(1)	TD Dense(16, ReLU) $\rightarrow$ Dense(1)	TD Dense(1)
Attention masking	Not applicable	Not applicable	Decoder self-attention masked or unmasked

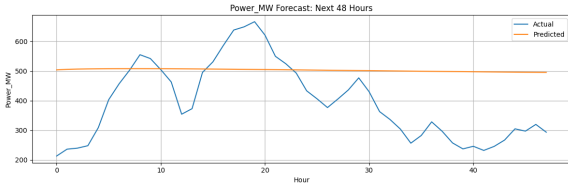


Figure 4: Ground truth versus predicted wind power for the GRU_Weak model over the forecast horizon. The predictions exhibit a strong bias toward average production levels, with limited responsiveness to abrupt changes in the true signal.

4.3 Training procedure

All deep learning models are trained using a unified training protocol to ensure comparability across architectures. Models are trained on the training split and validated on a held-out validation split using mini-batches of size 64. Early stopping based on the validation loss is employed to mitigate overfitting, while a `ReduceLROnPlateau` scheduler adapts the learning rate when validation performance stagnates. The model state achieving the best validation performance is preserved via checkpointing and subsequently evaluated on the test split.

Architecture-specific optimization strategies are applied for recurrent and Transformer-based models. Exact training hyperparameters and optimization settings for each model are summarized in Table 3.

5 Results

This section presents the forecasting results on the held-out test set. To keep the presentation concise, results are shown only for the first Norwegian bidding zone (NO2). All models are evaluated using identical temporal splits, input windows,

and a fixed 48-hour forecast horizon. Forecast quality is assessed quantitatively using RMSE and qualitatively through visual inspection of forecast trajectories.

It should be noted that the ground-truth trajectories shown across different figures are not always identical. This is because the figures illustrate different representative test windows sampled from the same test split within each bidding zone. While the specific time intervals shown therefore vary across plots, all quantitative metrics reported in this section are computed over the full test set and are directly comparable across models.

5.1 Statistical baseline models

SARIMA and SARIMAX serve as reference statistical baselines. Table 4 summarizes their quantitative performance for bidding zone NO2. Both models produce relatively smooth forecasts and struggle to capture rapid changes in wind power production. In particular, sharp ramps and sudden drops are underestimated, and predictions tend to revert toward a mean level.

Figure 5 and Figure 6 illustrate representative forecasts produced by the SARIMA and SARIMAX models, respectively. The limited responsiveness of these approaches highlights the difficulty of modeling complex wind dynamics using purely autoregressive structures.

5.2 Deep learning models

Deep learning models substantially improve forecasting performance compared to the statistical baselines, as shown in Table 4. Both recurrent and Transformer-based architectures are better able to capture daily seasonality and respond to changes in wind conditions.

Table 3: Training hyperparameters and optimization settings for deep learning models.

Parameter	GRU_Weak	GRU_Deep	Transformer
Optimizer	Adam	Adam	AdamW
Learning rate	2×10^{-4}	1×10^{-3}	1×10^{-3}
Batch size	64	64	64
Gradient clipping	1.0	1.0	—
L2 regularization	Strong	Moderate	—
Weight decay	—	—	1×10^{-5}
LR scheduler	ReduceLROnPlateau	ReduceLROnPlateau	ReduceLROnPlateau
Early stopping	Yes	Yes	Yes
Teacher forcing	—	—	Yes
Max epochs	100	100	100

Table 4: Test-set forecasting performance (RMSE in MW) for bidding zone NO2. Lower values indicate better performance.

Model	RMSE (MW)
SARIMA	138.6
SARIMAX	131.9
GRU_Weak	158.6
GRU_Deep	112.4
Transformer (masked decoder)	81.9
Transformer (unmasked decoder)	72.4

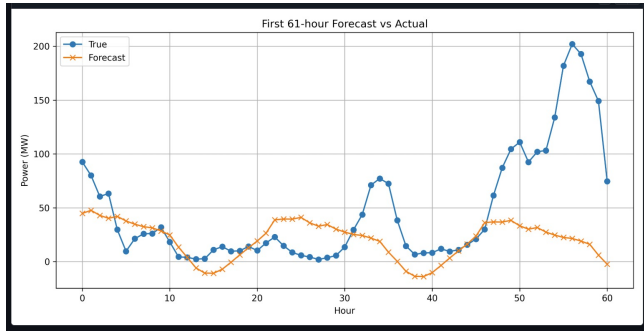


Figure 5: Forecast produced by the SARIMA model for bidding zone NO2.

Among the recurrent models, GRU_Deep achieves lower error than GRU_Weak. However, the GRU_Weak architecture frequently collapses toward near-constant predictions close to the empirical mean of the signal. This behavior is visible in ground-truth versus prediction plots (Figure 4) and results in large systematic error. Such behavior is consistent with recurrent models minimizing loss by regressing toward the central tendency of the data under limited effective capacity.

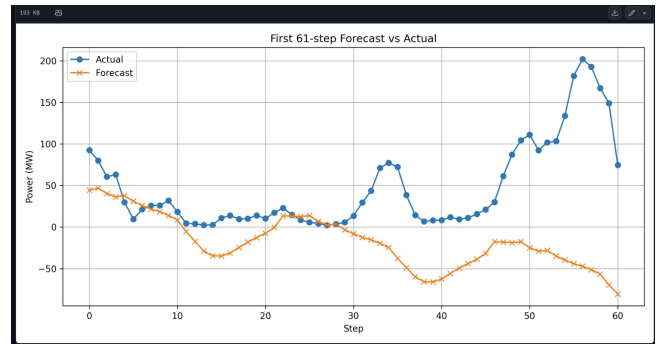


Figure 6: Forecast produced by the SARIMAX model for bidding zone NO2.

5.3 Transformer models: masked vs. unmasked decoder

Two Transformer variants are evaluated: (i) an unmasked decoder with unrestricted self-attention and (ii) a masked (causal) decoder constrained to attend only to past time steps. Both models share the same encoder architecture, input features, and training procedure; the only difference lies in the decoder self-attention mechanism.

As summarized in Table 4, both Transformer variants substantially outperform the statistical and recurrent baselines. The unmasked Transformer achieves the lowest RMSE overall, followed by the masked (causal) variant.

Figure 7 shows the forecast produced by the unmasked Transformer. The model closely tracks observed wind power production, capturing diurnal patterns, sharp ramps, and recovery phases across the full 48-hour forecast horizon.

Figure 8 presents the forecast from the masked Transformer. While the overall structure is similar, the causal constraint leads to slightly smoother predictions and reduced responsiveness during periods of rapid change.

Overall, the unmasked Transformer achieves the best forecasting performance in this study and is therefore selected for subsequent analysis. Ground-truth versus prediction visualizations are included in the Methods section for the GRU_Weak model to illustrate its tendency to converge toward the mean

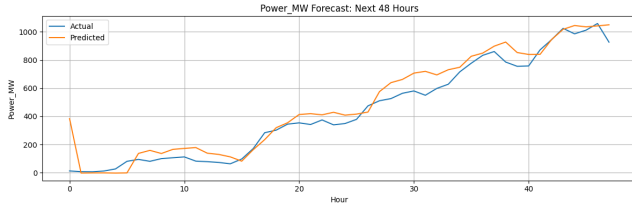


Figure 7: Forecast produced by the Transformer model with an unmasked decoder for bidding zone NO2.

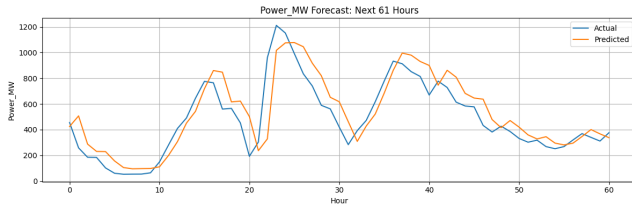


Figure 8: Forecast produced by the Transformer model with a masked (causal) decoder for bidding zone NO2.

or median of the wind power signal (Figure 4). Additional forecast plots for the GRU architectures are omitted here for brevity, as their predictive performance is substantially lower than that of the Transformer and their behavior is already sufficiently characterized by the diagnostic analyses presented earlier.

6 Discussion

The results show that Transformer-based models outperform both statistical models and recurrent neural networks for two-day-ahead wind power forecasting. This improvement is most noticeable at longer horizons.

Statistical models rely on strong assumptions about stationarity and linearity, which do not hold for wind power data. Recurrent models handle non-linearity better but struggle with long-range dependencies due to limited memory.

The Transformer performs better because attention allows the model to focus on relevant past time steps, even if they are far in the past. This matches the nature of wind dynamics, where delayed weather effects are common.

A limitation of this work is that we rely on a single, fixed data preparation pipeline. In particular, the feature engineering choices (time encodings, lag features, and ensemble summary statistics) are predefined and not systematically compared against alternative representations. Future work could explore richer feature engineering and preprocessing strategies, for example by applying signal decomposition methods such as Seasonal–Trend decomposition using Loess (STL)[6], wavelet transforms, or empirical mode decomposition to separate trend and seasonal components before training. Overall, this study suggests that attention-based models can perform well for operational wind power forecasting when combined with a clear temporal split and carefully designed input features.

Bibliography

- [1] NORA.ai, “Windai – forecasting wind power production.” <https://nora.ai/competitions/windai/windai-forecasting-wind-power-production>, 2025. Data Science Competition, Norway.
- [2] H. Zhou, S. Zhang, *et al.*, “Informer: Beyond efficient transformer for long sequence time-series forecasting,” *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021.
- [3] A. Zeng, M. Chen, L. Zhang, and Q. Xu, “Are transformers effective for time series forecasting?,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, pp. 11121–11128, AAAI Press, 2023.
- [4] K. Authen, S. Riemer-Sørensen, S. Razick, and K. Visoka, “Windai: Wind power forecasting in norway,” tech. rep., Nordic Machine Intelligence (NORA.ai), May 2025. Dataset description for the WindAI competition.
- [5] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019.
- [6] R. B. Cleveland, W. S. Cleveland, J. E. McRae, and I. Terpenning, “Stl: A seasonal-trend decomposition procedure based on loess,” *Journal of Official Statistics*, vol. 6, no. 1, pp. 3–73, 1990.